Reducing Cross-Node Network Calls in Distributed Payment Systems

B.Tech Project — End-Term Evaluation

with Concurrency-Safe Resolution of Single-Resource Contention

Raman Kapoor

ABV — Indian Institute of Information Technology and Management, Gwalior

Supervisor: Dr. Rahul Kala

2026

Outline

The Problem We Started With

Distributed payments at scale

- $\sim$68,000 cross-node operations per second
- Each request triggers **5–15 cross-node calls**
- p95 = 98 ms, p99 = 285 ms at peak
- Strict correctness: every transaction must be exactly once

The Problem We Started With

Distributed payments at scale

- $\sim$68,000 cross-node operations per second
- Each request triggers **5–15 cross-node calls**
- p95 = 98 ms, p99 = 285 ms at peak
- Strict correctness: every transaction must be exactly once

Tension

Aggressive caching reduces latency **but** risks staleness, lost updates, and double-allocation in a financial system.

What the Minor Established

- Two-tier in-process caching (thread + server-local) is **safe** for configuration-class data
- Formal cache-safety boundary framework (4 criteria)
- Phase 1 results in production:
  - **4.1%** cross-node call reduction at peak
  - **−11.2%** p95, **−13.0%** p99 API latency
  - **−26.2%** Redis p99 latency
  - Zero correctness incidents

What the Minor Established

- Two-tier in-process caching (thread + server-local) is **safe** for configuration-class data
- Formal cache-safety boundary framework (4 criteria)
- Phase 1 results in production:
  - **4.1%** cross-node call reduction at peak
  - **−11.2%** p95, **−13.0%** p99 API latency
  - **−26.2%** Redis p99 latency
  - Zero correctness incidents

Open question raised at the minor evaluation

"If you cache state, how do you prevent two users from booking the same single piece?"

This is the Question We Answer

End-term contribution

A **contention layer** that composes with the cache and provably prevents double-allocation on single, indivisible resources — inventory units, reservation slots, account balances, gateway tokens.

Five complementary primitives:

1. Distributed locking with fencing tokens
2. Atomic decrement via Redis Lua scripts
3. Optimistic concurrency control (versioned cache)
4. PN-Counter CRDTs (eventually consistent)
5. Single-flight request coalescing

Two-Tier Cache — Quick Recap

Tier 1: Thread-level

- `IORef HashMap`, per-request
- Destroyed on request end
- **Zero staleness** guarantee
- Hit rate: 22.3%

Tier 2: Server-local

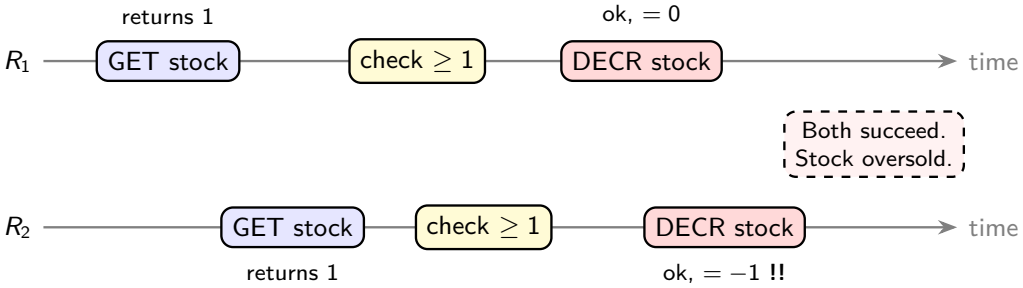
- `TVar` + LRU + TTL
- Shared across threads on a node
- TTLs: 30–120 s by data class
- Hit rate: 41.7%

Cache-safety boundaries (must hold for cache-eligibility)

1. Immutability within TTL
2. Request-scope independence
3. Idempotency of stale reads
4. Deterministic key

Why Caching Alone Is Not Enough

Single-resource state fails criterion (3): a stale read can lead to double-allocation.



The contention layer must close every variant of this race.

Primitive 1 — Distributed Locking with Fencing Tokens

- Acquire a per-resource lock (Redlock on Redis, etcd lease) before read-modify-write
- **Fence token:** monotonically increasing number returned with the lock; backend rejects stale-token writes
- Mitigates GC pauses, partitions, clock drift

Sketch

```
withLock conn key ttl $ \tok -> do
  v   <- backendRead key
  v'  <- compute v
  backendWriteIfFence key v' tok    -- backend checks tok
```

Use when: business logic is non-trivial; contention is moderate; the critical section calls external services.

Primitive 2 — Atomic Decrement via Redis Lua

Idea: package the entire "check + decrement" in one Lua script. Redis runs scripts atomically.

Lua

```lua
local a = tonumber(redis.call('GET', KEYS[1]) or '0')
local w = tonumber(ARGV[1])
if a >= w then
  redis.call('DECRBY', KEYS[1], w)
  return {1, a - w}
else
  return {0, a}
end
```

- Latency: <1 ms; throughput: $> 10^5$ scripts/s/shard
- Cache: **display reads** stay cached; **authoritative decrement** bypasses cache and publishes invalidation
- Limitation: critical section must fit in a Lua script (no external calls)

Primitive 3 — OCC with Versioned Cache Entries

- Each cache entry carries a version
- Reader records the version; writer issues a CAS (`WHERE version = ?`)
- Conflict $\Rightarrow$ retry

SQL form

```
UPDATE inventory
   SET stock = stock - 1, version = version + 1
 WHERE sku = $1 AND version = $2 AND stock >= 1
RETURNING stock, version;
```

Use when: contention is rare (e.g. control-plane writes).
Avoid when: contention is high — retry storms collapse throughput.

Primitive 4 — PN-Counter CRDTs

- Two grow-only counters per replica: P_i (positive), N_i (negative)
- Value $= \sum_i P_i - \sum_i N_i$; merge by element-wise max
- Writes are **local**, never blocked, eventually converge

When to use

Soft holds, cart abandonment counters, flash sales advertising "$\sim$1,000 units" — where a small overshoot is business-acceptable.

When not to use

Hard limits where every unit must be exact. CRDTs trade hard upper bounds for availability.

Primitive 5 — Single-Flight (Request Coalescing)

Idea: on a cache miss, only the **first** request fetches; concurrent requests for the same key wait on the leader's promise.

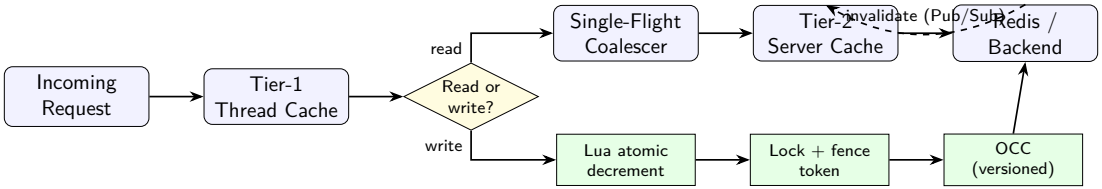
Effect

- Collapses the *front* of the contention curve
- One backend call instead of k identical ones
- Tames cache stampedes / thundering herds

Important

Single-flight **deduplicates** on a single node; it is **not** a substitute for atomicity. Compose it with Primitive 2 (Lua) for full safety across nodes.

Putting It Together



Reads go through cache + single-flight.
Authoritative writes bypass the cache and select a primitive per data class.

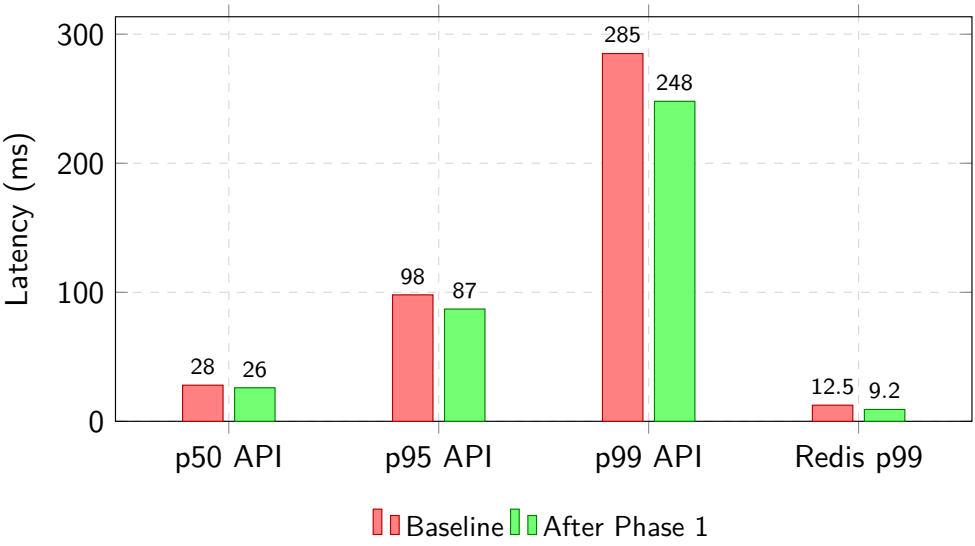
Decision Matrix

Data class	Cache?	Primary	Coalescing	Notes
Merchant config (read)	Tier-2	—	Single-flight	Cache-eligible
Routing table (read)	Tier-2	—	Single-flight	Cache-eligible
Feature flag (read)	Tier-2	—	Single-flight	Cache-eligible
Inventory (display)	Tier-2	—	Single-flight	Approximate
Inventory (decrement)	No	Lua atomic	Single-flight	Hard limit
Account balance debit	No	Distributed lock	Single-flight	Fence token
Reservation slot	No	Lua atomic	Single-flight	Per-slot key
Soft hold (cart)	Tier-2	PN-Counter	—	Over-alloc OK
Merchant config (write)	Inval.	OCC versioned	—	Low contention

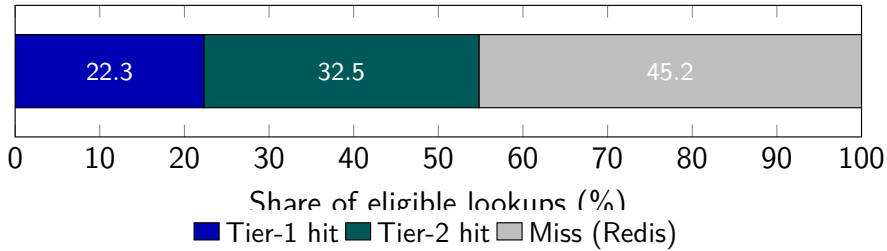
Comparison of the Five Primitives

Property	Lock	Lua	OCC	CRDT	Single-flight
Hard upper bound	Yes	Yes	Yes	No	N/A
Per-key throughput	Low	Very high	Medium–high	Very high	N/A
Latency overhead	1–3 ms	$<$1 ms	$<$1 ms	0 ms	0–1 ms
Behaviour under partition	Blocks/fails	Fails fast	Fails fast	Continues	Continues
Expressiveness	High	Lua only	SQL/CAS	Counter	N/A
Composes with cache	Bypass	Bypass + invalidate	Versioned cache	Cached	Front of cache

Tail-Latency Reductions (Phase 1)



Cache Hit-Rate Breakdown



- Combined hit rate **54.8%** on eligible lookups
- ∼2,800 cross-node calls/sec eliminated cluster-wide

Phase 2: Contention Layer in Production (30 days)

Workload	Primitive	Contention rate	p95 added
Inventory decrement (flash sale)	Lua + single-flight	1,400 contended/s	0.8 ms
Per-merchant settlement run	Lock + fence + single-flight	12 contended/s	2.6 ms
Reservation slot booking	Lua + single-flight	320 contended/s	1.1 ms
Cart soft hold	PN-Counter (no coord.)	N/A	0.0 ms

Correctness

- **Zero** double-allocation incidents
- **Zero** cache-related financial discrepancies
- Soft-hold over-allocation: 4.7 units/day on a 100k base ($<$1% tolerance)

Conclusion

What we built

An integrated two-layer system: a **caching layer** for read-side latency reduction and a **contention layer** that closes the single-resource race the minor evaluation surfaced.

- Cache-safety framework formalises which data is safe to cache
- Five contention primitives, each with a clear use case
- Decision matrix selects the right primitive per data class
- No new infrastructure beyond existing Redis cluster

Headline numbers

–11.2% p95, **–13.0% p99**, **–26.2% Redis p99**,
4.1% cross-node calls eliminated, **zero** correctness incidents.

Future Work

- **Adaptive TTLs** that shrink under detected mutation pressure
- **Learned contention prediction** — pre-warm the right primitive before flash sales
- **Verified fencing** — TLA$^+$ / Coq mechanisation of the safety argument
- **Cross-DC extension** — geo-replicated Redis vs. leader-elected coordinators
- **Graceful degradation modes** when Redis is partially unavailable

Selected References (2024+)

- Kumar et al., *Comparative Analysis of Distributed Caching Algorithms*, arXiv:2504.02220, 2024.
- Wang et al., *OptiQL: Robust Optimistic Locking for Memory-Optimised Indexes*, SIGMOD 2024.
- Wei et al., *Motor: Multi-Versioning for Distributed Transactions on Disaggregated Memory*, OSDI 2024.
- Zhou et al., *Concurrency Control as a Service*, VLDB 2025.
- *Distributed Locking: Performance Analysis and Optimisation Strategies*, arXiv:2504.03073, 2025.
- Duncan, *The CRDT Dictionary*, 2025.
- Redis Ltd., *Reserve Inventory in Real Time with Redis (WATCH/MULTI)*, 2024.

Thank You

Questions?

Raman Kapoor • ABV-IIITM Gwalior • 2026